Secure Student Onboarding Data Pipeline

GCP Infrastructure as Code • Poka-Yoke CI/CD • Django/DCYN Validation

Candidate: Gokul S
Role: Junior Cloud & DevOps Engineer

Terraform IaC
Provisioning & Access

Poka-Yoke CI/CD
Fail-Closed Build Gate

DCYN / DRF
Strict Validation

18/18
Python Tests Passed

3
IaC Categories

8
CI/CD Stages

5
Binary Fields Validated

Problem Statement & Engineering Objectives

Sensitive student onboarding data requires secure, auditable handling at every layer.

Identified Risks

- Inconsistent raw Boolean values (Yes / No / Pending / N/A / Maybe) cause ambiguous staging.
- Potential hardcoded secrets in source code expose credentials to version history.
- Over-permissive infrastructure access patterns violate least-privilege principles.
- Invalid or ambiguous records must not silently reach the staged analytics layer.

1. Secure Raw-Data Landing
Access-controlled GCS bucket with uniform permissions and public access prevention.

2. Strict Validation & Normalization
Reject ambiguous values at ingestion. No silent corrections or guesses.

3. Consent-Aware Data Access
BigQuery Row-Level Security based on `parent_consent_given`.

4. Automated Secret Scanning
Gitleaks blocks insecure commits before they can progress.

5. Terraform IaC + Fail-Closed Controls
Infrastructure is defined as code; required CI checks must pass.

End-to-End Architecture

Data Pipeline — Ingestion to Analytics

DATA PLANE

Raw JSON Payload → D0 Raw Landing (GCS Bucket) → Django / DCYN (Strict Validation) → D1 Staged Layer (BigQuery student_onboarding) → Authorized Analytics (Read-only + RLS)

CONTROL PLANE

Git Repository → GitHub Actions → Lint + Tests + TF Checks + Gitleaks → Fail-Closed Gate → Proceed / Quarantine

INFRASTRUCTURE PROVISIONING (TERRAFORM)

Terraform → GCS Bucket | BigQuery Dataset/Table | IAM Roles & Service Accounts | Row-Level Security

⚠ Terraform defines the GCP infrastructure and security controls; deployment was environment-blocked by GCP billing.

Secure Staging Provisioning with Terraform

Resource	Purpose
D0 Raw Landing BucketGoogle Cloud Storage	Uniform access, versioning, prefix restriction
D1 Staged Dataset & TableBigQuery	Enforced schema and consent-filtered data with Row-Level Security
Pipeline Service AccountIAM Identity	Least-privilege data processing
Analytics AccessIAM Role Binding	Read-only BigQuery Data Viewer access

```
(.venv) PS C:\Users\gokul\Downloads\Task\Terraform Secure Staging Provisioning (IaC)> terraform validate
Success! The configuration is valid.
```

```
$ terraform fmt          PASSED
$ terraform validate     PASSED
$ terraform plan         GENERATED
$ terraform apply        BLOCKED
```

```
Error: googleapi: Error 403: The billing account for the owning project is disabled in state closed, accountDisabled
```

Environment Prerequisite Notification

The Terraform configuration passed formatting, validation, and planning successfully. The apply step was blocked because the assigned GCP project's billing account was closed/disabled.

Note: This is an environment-level prerequisite block, distinctly separate from configuration or logical validation failures.

D0 — Secure Raw Landing

Incoming Raw Payload

↓

D0 Raw Landing Bucket
Google Cloud Storage

↓

Pipeline Service Account
Authorized Writer

↓

Django / DCYN
Validation

1. Uniform Bucket-Level Access
Consistent permission model; no per-object ACL overrides allowed.

2. Public Access Prevention
Public access strictly blocked at the resource level.

3. Object Versioning
Maintains change history for audit trails and rollback capabilities.

4. Lifecycle-Based Retention
Automatic cleanup after the defined staging/reprocessing window.

5. d0/ Prefix Restriction
Object creation restricted to the landing-zone prefix only.

Terraform defines a protected D0 landing-zone configuration with uniform access, public-access prevention, versioning, lifecycle retention, and prefix-restricted writes.

D1 — Validated Data & Consent Enforcement

Access Control Matrix

Pipeline Service Account
Role: BigQuery Data Editor
Capability: Write validated records

Analytics Users
Role: BigQuery Data Viewer
Capability: Read-only, RLS enforced

Unauthorized Identity
Role: No binding
Capability: Zero access

Data-Layer Consent Enforcement

```
parent_consent_given = true
```

BigQuery Row-Level Security filters visible records based on the consent flag.

- Policy is enforced at the data layer, completely independent of application-level permissions.
- Terraform configuration defines the consent-based Row-Level Security policy.

Automated Poka-Yoke Build Gate

Sequential CI/CD validation — no partial pass

1
Python Format
black / isort

2
Python Lint
flake8

3
Python Tests
18/18 pass required

4
Terraform Format
terraform fmt

5
Terraform Init
provider init

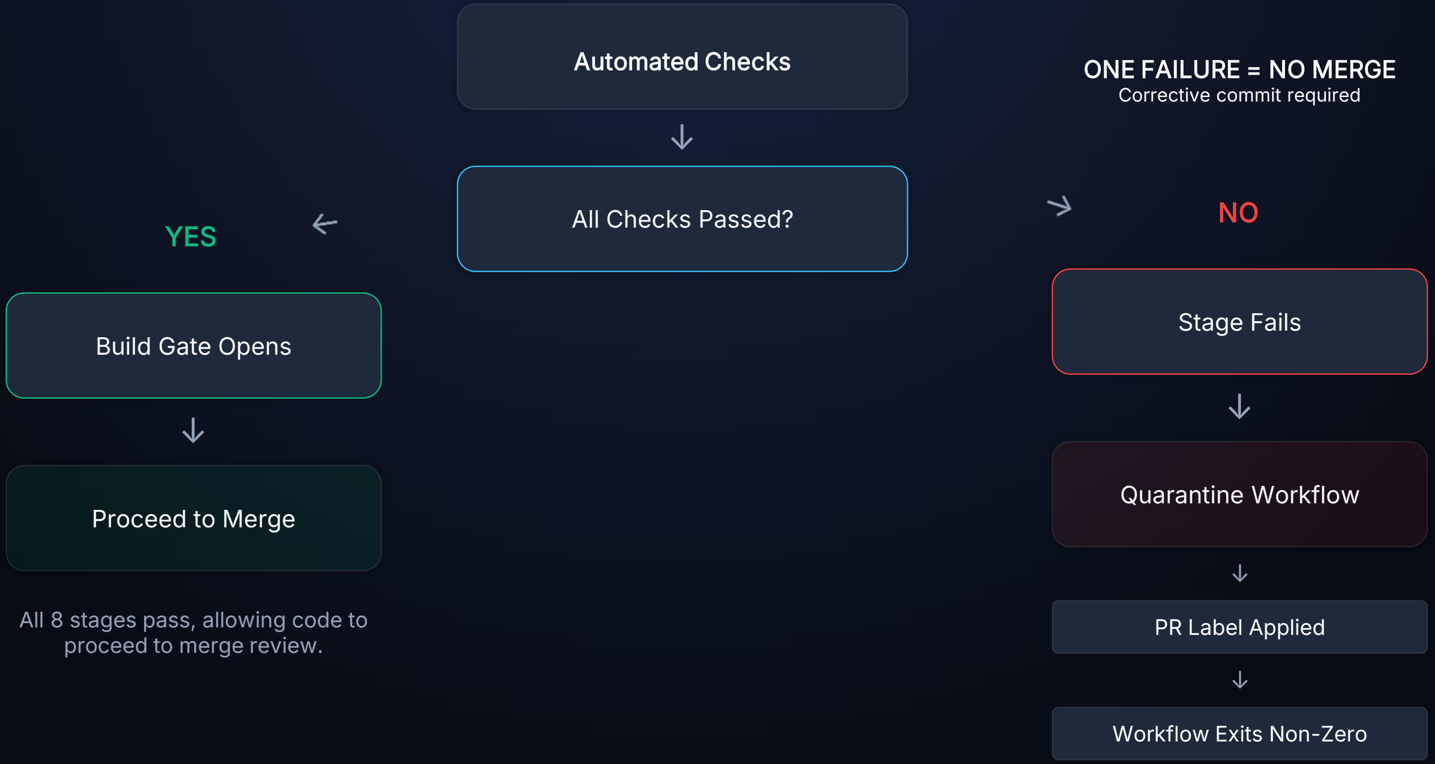
6
Terraform Validate
config check

7
Gitleaks Scan
secret detection

8
Fail-Closed Gate
all-or-nothing

Core Principle: The CI/CD gate fails closed when required checks fail. Every required prerequisite must succeed before the gate opens.

Fail-Closed Decision & Quarantine Flow



All 8 stages pass, allowing code to proceed to merge review.

Any single failure triggers quarantine, requiring a corrective commit.

Secret Detection — Gitleaks

Django SECRET_KEY Pattern

Custom regex rule detects Django SECRET_KEY patterns in code.

GCP Service Account Key Pattern

Detects GCP service-account private-key JSON structures.

Default Ruleset Retained

Common secret patterns (API keys, tokens, credentials) remain covered.

Full Git History Scanned

The scanner checks repository history, not only the latest diff.

Hardcoded Secret in Code

↓

Custom Gitleaks Rule Matches

↓

Scan Returns Failure

↓

Build Gate Remains Closed

```
gitleaks

8:56PM INF Overriding enabled rules: django-secret-key
8:56PM INF scanned ~84 bytes (84 bytes) in 3.52ms
8:56PM WRN leaks found: 1

Controlled test:
Django SECRET_KEY pattern injected

Gitleaks:
Finding detected
Scan FAILED
Exit code: 1

Result:
Build gate BLOCKED
```

DCYN + Django REST Framework Validation

Raw JSON Payload

Unvalidated onboarding submission.

→

Field Mapping

Schema alignment and normalization.

→

DCYN Strict Boolean

“No guessing / no silent coercion”

Only explicit Yes/No values are accepted.

→

DRF Validation

“Type + field constraints”

Serializer-level type and constraint validation.

→

Business Rules

Cross-field logic validation.

→

D1 BigQuery

Only validated records are written to the staged layer.

DCYN Strict Boolean Normalization

Input Value	DCYN Result
"Yes"	→ True
"No"	→ False
"Pending"	→ REJECT
"N/A"	→ REJECT
"Maybe"	→ REJECT
Empty / null	→ REJECT

`INVALID ≠ FALSE`

> "Ambiguous values are rejected rather than guessed or silently corrected."

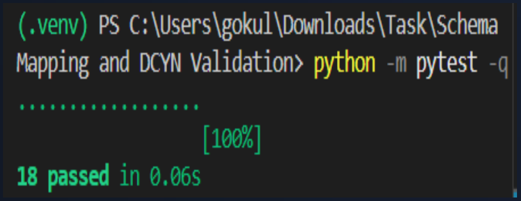
Five Binary Fields Validated per Record:

- → `has_existing_learning_diagnosis`
- → `diagnosis_documentation_uploaded`
- → `requires_learning_support_assistant`
- → `parent_consent_statement_accepted`
- → `guardian_identity_verified`

Invalid records are caught and rejected during the Django Serializer validation phase before they can ever enter the D1 BigQuery staged layer.

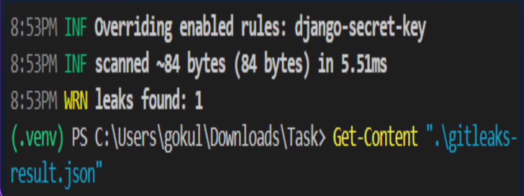
Testing & Engineering Validation

Python Tests Passed

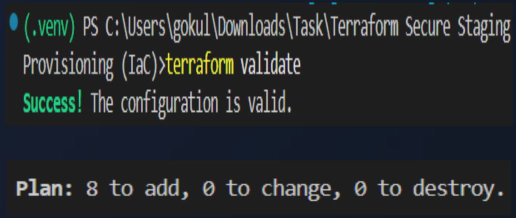


Gitleaks Custom Rule

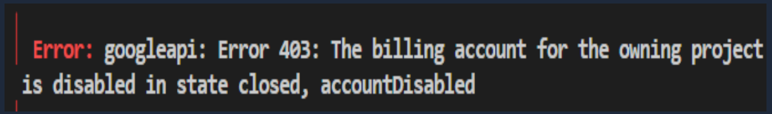
SECRET_KEY Pattern



Terraform Validation Stages



terraform fmt	PASSED
terraform validate	PASSED
terraform plan	GENERATED
terraform apply	BLOCKED



Environment Prerequisite Block

terraform apply could not proceed because the GCP billing account was disabled.

This was an environment prerequisite constraint enforcing infrastructure policies, **not a Terraform configuration validation failure.**

Engineering Outcomes

SECURITY

Secret scanning, least-privilege IAM configurations, and BigQuery Row-Level Security enforcement at the data layer.

RELIABILITY

Fail-closed CI/CD pipeline prevents broken or insecure code from progressing through the required checks.

INFRASTRUCTURE

Reproducible, version-controlled GCP architecture fully defined using Terraform Infrastructure as Code.

DATA QUALITY

Strict schema definition, DCYN Boolean normalization, and business-rule enforcement prior to staging.

KEY TAKEAWAY: Security, automation, infrastructure-as-code, and data-quality controls work together to create a fail-closed onboarding pipeline from ingestion through analytics.

Thank You